Solving Linear Systems

September 2026

1 Introduction

Many problems in numerical analysis, engineering, and scientific computing can be expressed as a system of linear equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

This system can be written compactly in matrix form as

$$A\mathbf{x} = \mathbf{b},$$

where

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}.$$

Here,

- A is the **coefficient matrix**,
- $\mathbf{x}$ is the **unknown vector**,
- $\mathbf{b}$ is the **right-hand-side vector**.

The objective is to determine $\mathbf{x}$ such that $A\mathbf{x} = \mathbf{b}$.

1.1 Direct and Iterative Methods

Direct methods obtain the solution through a finite sequence of arithmetic operations. In exact arithmetic, they produce the exact solution after a finite number of steps. Methods include Gaussian Elimination and LU Decomposition.

Iterative methods begin with an initial approximation $\mathbf{x}^{(0)}$ and repeatedly improve it

$$\mathbf{x}^{(0)} \to \mathbf{x}^{(1)} \to \mathbf{x}^{(2)} \to \cdots$$

until the approximation satisfies a chosen convergence criterion. Methods include Jacobi Method, Gauss-Seidel Method and Conjugate Gradient Method.

2 Gaussian Elimination

Gaussian Elimination is a **direct method** for solving a system of linear equations $A\mathbf{x} = \mathbf{b}$. The main idea is to transform the original system into an **upper triangular system**, which can then be solved using **back substitution**.

2.1 Forward Elimination

Consider the system

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 8, \\ -3x_1 - x_2 + 2x_3 &= -11, \\ -2x_1 + x_2 + 2x_3 &= -3. \end{aligned}$$

Its augmented matrix is

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array}\right].$$

The goal of forward elimination is to eliminate the elements below the diagonal, transforming the matrix into the form

$$\left[\begin{array}{ccc|c} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \end{array}\right].$$

Pivot :
The **first pivot** is $a_{11} = 2$. To eliminate $a_{21} = -3$, calculate the multiplier

$$m_{21} = \frac{a_{21}}{a_{11}} = \frac{-3}{2}.$$

Then perform the row operation

$$R_2 - m_{21}R_1 \to R_2.$$

Similarly, to eliminate $a_{31} = -2$,

$$m_{31} = \frac{a_{31}}{a_{11}} = \frac{-2}{2} = -1,$$

and

$$R_3 - m_{31}R_1 \to R_3.$$

This produces

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 2 & 1 & 5 \end{array}\right].$$

The **second pivot** is $a_{22} = \frac{1}{2}$. To eliminate $a_{32} = 2$, calculate

$$m_{32} = \frac{a_{32}}{a_{22}} = \frac{2}{1/2} = 4.$$

Then perform

$$R_3 - m_{31}R_2 \to R_3.$$

We obtain

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 0 & -1 & 1 \end{array}\right].$$

The coefficient matrix is now in **upper triangular form**.

2.2 Back Substitution

The resulting system is

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 8, \\ \frac{1}{2}x_2 + \frac{1}{2}x_3 &= 1, \\ -x_3 &= 1. \end{aligned}$$

Starting from the last equation,

$$x_3 = -1.$$

Substitute $x_3 = -1$ into the second equation:

$$\frac{1}{2}x_2 + \frac{1}{2}(-1) = 1,$$

which gives

$$x_2 = 3.$$

Finally, substitute $x_2 = 3$ and $x_3 = -1$ into the first equation:

$$2x_1 + 3 - (-1) = 8,$$

so

$$x_1 = 2.$$

Therefore,

$$\boxed{\mathbf{x} = \begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix}}.$$

2.3 Partial Pivoting

During forward elimination, the multiplier is calculated as

$$m_{ik} = \frac{a_{ik}}{a_{kk}},$$

where a_{kk} is the current **pivot**. If the pivot $a_{kk} = 0$, the multiplier cannot be calculated. Furthermore, if a_{kk} is very close to zero, division by a small number may amplify round-off errors and lead to numerical instability.

To avoid this problem, Gaussian Elimination is commonly implemented with **partial pivoting**. For each pivot column k, partial pivoting selects the element with the largest absolute value among the entries at or below the current pivot position:

$$|a_{pk}| = \max_{i=k,\ldots,n} |a_{ik}|.$$

If $p \neq k$, rows k and p are swapped:

$$R_k \leftrightarrow R_p.$$

For example, consider

$$\left[\begin{array}{cc|c} 0.001 & 1 & 1 \\ 5 & 2 & 7 \\ -2 & 3 & 1 \end{array}\right].$$

In the first column,

$$|5| > |-2| > |0.001|,$$

so the second row is selected as the pivot row. After interchanging R_1 and R_2,

$$\left[\begin{array}{cc|c} 5 & 2 & 7 \\ 0.001 & 1 & 1 \\ -2 & 3 & 1 \end{array}\right].$$

The elimination can then proceed using 5 as the first pivot. Partial pivoting improves the numerical stability of Gaussian Elimination and avoids division by zero when a suitable nonzero pivot exists.

2.4 General Algorithm

For a general system

$$A\mathbf{x} = \mathbf{b},$$

Gaussian Elimination with partial pivoting consists of **forward elimination** followed by **back substitution**.

Forward Elimination

For each pivot column $k = 1, 2, \ldots, n-1$, perform the following steps.

1. **Select the pivot row**

   Find the row p containing the largest absolute value in the current pivot column:

   $$p = \underset{i=k,\ldots,n}{\operatorname{argmax}} |a_{ik}|.$$

2. **Check the pivot**

   If $|a_{pk}| < \varepsilon$, the matrix is singular or nearly singular, and the elimination cannot safely continue.

3. **Swap rows**

   If $p \neq k$, interchange rows k and p: $R_k \leftrightarrow R_p$. The corresponding elements of the right-hand-side vector $\mathbf{b}$ must also be interchanged.

4. **Eliminate the entries below the pivot**

   For $i = k+1, \ldots, n$, calculate the multiplier $m_{ik} = \frac{a_{ik}}{a_{kk}}$.

   Then update the elements in row i:

   $$a_{ij} - m_{ik}a_{kj} \to a_{ij}, \qquad j = k, k+1, \ldots, n,$$

   and update the corresponding element of the right-hand-side vector:

   $$b_i - m_{ik}b_k \to b_i.$$

After forward elimination, the coefficient matrix is transformed into an upper triangular matrix.

*** Before performing back substitution, the final pivot must also be checked. If $|a_{nn}| < \varepsilon$, the matrix is singular or nearly singular.

Back Substitution

Starting from the last equation,

$$x_n = \frac{b_n}{a_{nn}}.$$

The remaining unknowns are calculated from bottom to top:

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}}, \qquad i = n-1, n-2, \ldots, 1.$$

3 LU Decomposition

Using Gaussian elimination, we eliminate the entries below the diagonal.

$$A = \begin{bmatrix} 3 & -1 & 2 \\ 6 & -1 & 5 \\ -9 & 7 & 3 \end{bmatrix}$$

Let $l_{21} = \frac{6}{3} = 2$, then we obtain

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ -9 & 7 & 3 \end{bmatrix}.$$

Next, let $l_{31} = \frac{-9}{3} = -3$, which gives

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 4 & 9 \end{bmatrix}.$$

Finally, $l_{32} = \frac{4}{1} = 4$, so that

$$\begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 5 \end{bmatrix}.$$

The elimination multipliers are stored in the **lower triangular matrix**:

$$L = \begin{bmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -3 & 4 & 1 \end{bmatrix}.$$

Therefore,

$$\boxed{A = LU}$$

where

$$\boxed{\begin{bmatrix} 3 & -1 & 2 \\ 6 & -1 & 5 \\ -9 & 7 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -3 & 4 & 1 \end{bmatrix} \begin{bmatrix} 3 & -1 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 5 \end{bmatrix}}$$

The entries of L are the elimination multipliers used during Gaussian elimination. The diagonal entries of L are all equal to 1 because each row retains its original leading coefficient during the elimination process.

The main advantage of LU decomposition is that it allows us to solve a linear system efficiently **when the same coefficient matrix is used with multiple right-hand sides.** Given $Ax = b$, and the decomposition $A = LU$, we obtain $LUx = b$ and $Ux = y$.

Then the system can be solved in two triangular systems: $Ly = b$, followed by $Ux = y$. The first system is solved by **forward substitution**, while the second is solved by **back substitution**.

3.1 LU Decomposition with Partial Pivoting

The previous example assumes that no row interchanges are required. However, in practical numerical computation, partial pivoting may be necessary for the same reasons discussed in Gaussian elimination. Consider

$$A = \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix}.$$

The first pivot is $a_{11} = 0$. Since the pivot is zero, we cannot perform the elimination directly. Using partial pivoting, we select the largest absolute value in the first column:

$$|2| > |1| > |0|.$$

Therefore, the third row is selected as the pivot row. We interchange R_1 and R_3:

$$R_1 \leftrightarrow R_3,$$

giving

$$\begin{bmatrix} 2 & 1 & 1 \\ 1 & 1 & 2 \\ 0 & 2 & 1 \end{bmatrix}.$$

The first pivot is now $a_{11} = 2$. The elimination multipliers are

$$l_{21} = \frac{1}{2}, \qquad l_{31} = \frac{0}{2} = 0.$$

Thus,

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 2 & 1 \end{bmatrix}.$$

The second pivot is $a_{22} = \frac{1}{2}$. Therefore,

$$l_{32} = \frac{2}{1/2} = 4.$$

Performing the elimination, we obtain

$$U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 0 & -5 \end{bmatrix}.$$

The elimination multipliers are stored in

$$L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & 4 & 1 \end{bmatrix}.$$

Because we interchanged the first and third rows, we also need a permutation matrix

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

The row interchange means that we decomposed PA, rather than A:

$$\boxed{PA = LU}$$

Indeed,

$$\boxed{\begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 0 & 4 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & \frac{1}{2} & \frac{3}{2} \\ 0 & 0 & -5 \end{bmatrix}}$$

3.2 Solving Using LU Decomposition

Once the decomposition $PA = LU$ has been computed, solving $A\mathbf{x} = \mathbf{b}$ does not require performing Gaussian elimination again. Multiplying both sides by the permutation matrix gives $PA\mathbf{x} = P\mathbf{b}$. Since $PA = LU$, we obtain $LU\mathbf{x} = P\mathbf{b}$.

Rather than solving this system directly, we introduce an intermediate vector $\mathbf{y}$ and solve two triangular systems: $L\mathbf{y} = P\mathbf{b}$ followed by $U\mathbf{x} = \mathbf{y}$. The first system is solved by forward substitution, while the second is solved by back substitution.

Forward substitution :

Because L is lower triangular with ones on the diagonal, the unknowns are computed from top to bottom. For the system $L\mathbf{y} = P\mathbf{b}$, the equations are

$$\begin{aligned} y_1 &= (P\mathbf{b})_1, \\ y_2 &= (P\mathbf{b})_2 - l_{21}y_1, \\ y_3 &= (P\mathbf{b})_3 - l_{31}y_1 - l_{32}y_2, \\ &\vdots \\ y_n &= (P\mathbf{b})_n - \sum_{i=1}^{n-1} l_{ni}y_i. \end{aligned}$$

In general,

$$y_i = (P\mathbf{b})_i - \sum_{j=1}^{i-1} l_{ij}y_j, \qquad i = 1, 2, \ldots, n.$$

Since the diagonal entries of L are all equal to 1, no division is required.

Back substitution :

After obtaining y, we solve $U\mathbf{x} = \mathbf{y}$. Because U is upper triangular, the unknowns are computed from bottom to top. Starting from the last equation,

$$x_n = \frac{y_n}{u_{nn}}.$$

The remaining unknowns are calculated from bottom to top:

$$x_i = \frac{y_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}, \qquad i = n-1, n-2, \ldots, 1.$$

Unlike forward substitution, division by the diagonal entries of U is necessary because they are generally not equal to one.

Regardless of coding, the lower-left entries store the multipliers of L, while the diagonal and upper-right entries store U. The diagonal of L is implicitly equal to 1. Thus, the matrix could be like

$$\begin{bmatrix} U & U & U \\ L & U & U \\ L & L & L \end{bmatrix}$$

So instead of allocating another $n \times n$ matrix, we overwrite those unused positions. This saves roughly half the memory while still allowing us to reconstruct both matrices whenever we need them.

4 Jacobi Method

The Jacobi Method, also known as the **Jacobi Iterative Method**, is a fundamental algorithm used to solve systems of linear equations $A\mathbf{x} = \mathbf{b}$.

Unlike Gaussian Elimination or LU Decomposition, which compute the solution through direct factorization, the Jacobi Method begins with an initial guess and repeatedly improves the approximation until it converges to the solution. Because each component of the new iterate is computed independently from the previous iterate, the method is easy to implement and naturally suitable for parallel computation.

4.1 Basic of Jacobi Method

For linear system $A\mathbf{x} = \mathbf{b}$, suppose the coefficient matrix is written as

$$A = D + L + U,$$

where D is the diagonal matrix containing the diagonal entries of A, L contains the entries below the diagonal, and U contains the entries above the diagonal.

$$D = \begin{bmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{bmatrix}, \qquad L = \begin{bmatrix} 0 & 0 & \dots & 0 \\ a_{21} & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ a_{n1} & \dots & a_{n,n-1} & 0 \end{bmatrix}, \qquad U = \begin{bmatrix} 0 & a_{12} & \dots & a_{1n} \\ 0 & 0 & \ddots & \vdots \\ \vdots & \ddots & \ddots & a_{n-1,n} \\ 0 & 0 & \dots & 0 \end{bmatrix}.$$

Substituting this decomposition into the original system gives

$$(D + L + U)\mathbf{x} = \mathbf{b}.$$

Rearranging,

$$D\mathbf{x} = \mathbf{b} - (L + U)\mathbf{x}.$$

Since D is diagonal, it is easy to invert, yielding the Jacobi iteration formula

$$\mathbf{x}^{(k+1)} = D^{-1}(\mathbf{b} - (L + U)\mathbf{x}^{(k)}).$$

4.2 Derive the component-wise formula

To derive the component-wise iteration formula, consider the linear system

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6, \\ -2x_1 + 5x_2 + 2x_3 &= 6, \\ 2x_1 - x_2 + 10x_3 &= -10. \end{aligned}$$

The coefficient matrix is

$$A = \begin{bmatrix} 10 & -1 & 2 \\ -2 & 5 & 2 \\ 2 & -1 & 10 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 6 \\ 6 \\ -10 \end{bmatrix}.$$

Notice that the matrix is **strictly diagonally dominant**, so the Jacobi Method is expected to converge.

The coefficient matrix can be decomposed as $A = D + L + U$, where

$$D = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 10 \end{bmatrix}, \qquad L = \begin{bmatrix} 0 & 0 & 0 \\ -2 & 0 & 0 \\ 2 & -1 & 0 \end{bmatrix}, \qquad U = \begin{bmatrix} 0 & -1 & 2 \\ 0 & 0 & 2 \\ 0 & 0 & 0 \end{bmatrix}.$$

Based on the Jacobi iteration formula we can write

$$\begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 10 \end{bmatrix} \left(\begin{bmatrix} 6 \\ 6 \\ -10 \end{bmatrix} - \begin{bmatrix} 0 & -1 & 2 \\ -2 & 0 & 2 \\ 2 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix} \right).$$

Expanding the matrix equation gives

$$\begin{aligned} x_1^{(1)} &= \frac{1}{10}(6 - (-x_2^{(0)} + 2x_3^{(0)})), \\ x_2^{(1)} &= \frac{1}{5}(6 - (-2x_1^{(0)} + 2x_3^{(0)})), \\ x_3^{(1)} &= \frac{1}{10}(-10 - (2x_1^{(0)} - x_2^{(0)})). \end{aligned}$$

Thus, the Jacobi iteration is

$$\begin{aligned} x_1^{(k+1)} &= \frac{(6 + x_2^{(k)} - 2x_3^{(k)})}{10}, \\ x_2^{(k+1)} &= \frac{(6 + 2x_1^{(k)} - 2x_3^{(k)})}{5}, \\ x_3^{(k+1)} &= \frac{(-10 - 2x_1^{(k)} + x_2^{(k)})}{10}. \end{aligned}$$

Therefore, for a general $n \times n$ system, the Jacobi iteration is

$$\boxed{x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{i \neq j} a_{ij} x_j^{(k)}).}$$

4.3 Worked Example

From previous example we have

$$\begin{aligned} x_1^{(k+1)} &= \frac{(6 + x_2^{(k)} - 2x_3^{(k)})}{10}, \\ x_2^{(k+1)} &= \frac{(6 + 2x_1^{(k)} - 2x_3^{(k)})}{5}, \\ x_3^{(k+1)} &= \frac{(-10 - 2x_1^{(k)} + x_2^{(k)})}{10}. \end{aligned}$$

Choose the initial guess

$$\mathbf{x^{(0)}} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Iteration 1 : Using $\mathbf{x^{(0)}}$

$$\begin{aligned} x_1^{(1)} &= \frac{(6+0-2(0))}{10} = 0.600 \\ x_2^{(1)} &= \frac{(6+0-0)}{5} = 1.200 \\ x_3^{(1)} &= \frac{(-10-2(0)+0)}{10} = -1.000 \end{aligned} \qquad \rightarrow \mathbf{x^{(1)}} = \begin{bmatrix} 0.6000 \\ 1.2000 \\ -1.0000 \end{bmatrix}$$

Iteration 2 : Using $\mathbf{x^{(1)}}$

$$\begin{aligned} x_1^{(2)} &= \frac{(6+1.2000-2(-1.0000))}{10} = 0.9200 \\ x_2^{(2)} &= \frac{(6+2(0.6000)-2(-1.0000))}{5} = 1.8400 \\ x_3^{(2)} &= \frac{(-10-2(0.6000)+1.2000)}{10} = -1.0000 \end{aligned} \qquad \rightarrow \mathbf{x^{(2)}} = \begin{bmatrix} 0.9200 \\ 1.8400 \\ -1.0000 \end{bmatrix}$$

The approximations gradually approach the exact solution

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}.$$

4.4 Convergence of the Jacobi Method

Recall the matrix splitting $A = D + L + U$, the Jacobi iteration is

$$\mathbf{x}^{(k+1)} = D^{-1}\mathbf{b} - D^{-1}(L+U)\mathbf{x}^{(k)}.$$

Define the iteration matrix $T = -D^{-1}(L+U)$. Then the iteration becomes

$$\mathbf{x}^{k+1} = D^{-1}\mathbf{b} + T\mathbf{x}^{(k)}.$$

The behavior of the method is completely determined by the matrix T.

The Jacobi Method converges if the spectral radius of the iteration matrix satisfies $\rho(T) < 1$. Here, $\rho(T) = \max_i |\lambda_i|$ is the largest absolute value among all eigenvalues of T.

The Jacobi iteration can be written as $\mathbf{x}^{k+1} = \mathbf{c} + T\mathbf{x}^{(k)}$, where T is the iteration matrix, $\mathbf{c}$ is a constant vector, and $\mathbf{x}^{(k)}$ is the approximation after the k-th iteration.

Suppose the exact solution is $\mathbf{x}^*$. Since it satisfies the same iteration equation

$$\mathbf{x}^* = \mathbf{c} + T\mathbf{x}^*.$$

Define the error at the k-th iteration as

$$\mathbf{e^{(k)}} = \mathbf{x}^{(k)} - \mathbf{x}^*.$$

Subtracting the two equations eliminates the constant vector $\mathbf{c}$, giving

$$\begin{aligned} \mathbf{e^{(k+1)}} &= \mathbf{x}^{(k+1)} - \mathbf{x}^* \\ &= (\mathbf{c} + T\mathbf{x}^{(k)}) - (\mathbf{c} + T\mathbf{x}^*) \\ &= T(\mathbf{x}^{(k)} - \mathbf{x}^*) \\ &= T\mathbf{e^{(k)}} \end{aligned}$$

This recurrence shows that the error is transformed by the iteration matrix at each step. Applying the recurrence repeatedly gives

$$\begin{aligned}
\mathbf{e^{(1)}} &= T\mathbf{e}^{(0)} \\
\mathbf{e^{(2)}} &= T\mathbf{e}^{(1)} = T^2\mathbf{e}^{(0)} \\
&\vdots \\
\mathbf{e}^{(k)} &= T^{(k-1)}\mathbf{e}^{(k-1)} = T^k\mathbf{e^{(0)}}
\end{aligned}$$

Assume that the iteration matrix T has n linearly independent eigenvectors

$$\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_n,$$

with corresponding eigenvalues

$$\lambda_1, \lambda_2, \ldots, \lambda_n.$$

Since these eigenvectors form a basis of the vector space, any initial error can be written as a linear combination of them

$$\mathbf{e^{(0)}} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n,$$

where $c_1, c_2, \ldots, c_n$ are constants.

By the definition of an eigenvalue $T\mathbf{v}_i = \lambda\mathbf{v}_i$. Applying the matrix repeatedly gives

$$T^k\mathbf{v}_i = \lambda^k\mathbf{v}_i.$$

Substituting this into the error formula,

$$\begin{aligned}
\mathbf{e}^{(k)} &= T^k\mathbf{e^{(0)}} \\
&= c_1(T^k\mathbf{v}_1) + c_2(T^k\mathbf{v}_2) + \cdots + c_n(T^k\mathbf{v}_n) \\
&= c_1\lambda^k\mathbf{v}_1 + c_2\lambda^k\mathbf{v}_2 + \cdots + c_n\lambda^k\mathbf{v}_n.
\end{aligned}$$

Now consider the limit as $k \to \infty$. If every eigenvalue satisfies

$$|\lambda_i| < 1, \qquad i = 1, 2, \ldots, n.$$

then each term λ_i^k tends to zero $\lim_{k\to\infty} \lambda_i^k = 0$.

Consequently,

$$\lim_{k\to\infty} e^{(k)} = 0.$$

The approximate solution converges to the exact solution $\lim_{k\to\infty} x^{(k)} = x^*$.Therefore, the Jacobi Method converges whenever all eigenvalues of the iteration matrix lie inside the unit circle, which is equivalent to the condition

$$\rho(T) = \max_i |\lambda_i| < 1.$$

4.5 Strictly diagonally dominant matrix

Let A be a strictly diagonally dominant matrix, meaning that

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|, \qquad i = 1, 2, \ldots, n.$$

Let λ be an arbitrary eigenvalue of T, with corresponding eigenvector $\mathbf{v} \neq 0$. By the definition of an eigenvalue, $T\mathbf{v} = \lambda\mathbf{v}$. Substituting $T = -D^{-1}(L + U)$ gives

$$-D^{-1}(L + U)\mathbf{v} = \lambda\mathbf{v}.$$

Multiplying both sides by D yields

$$-(L + U)\mathbf{v} = \lambda D\mathbf{v}.$$

For the i-th row, this becomes

$$-\sum_{j \neq i} a_{ij} v_j = \lambda a_{ii} v_i.$$

Now choose the component of v with the largest magnitude. Suppose it is the m-th component, so that

$$|v_m| = \max_{1 \leq j \leq n} |v_j| > 0.$$

Applying the previous equation to the m-th row,

$$\lambda a_{mm} v_m = -\sum_{j \neq m} a_{mj} v_j$$

Taking absolute values and applying the triangle inequality,

$$|\lambda||a_{mm}||v_m| = |\sum_{j \neq m} a_{mj} v_j| \leq \sum_{j \neq m} |a_{mj}||v_j|.$$

Since v_m is the largest component in magnitude, therefore

$$|\lambda||a_{mm}||v_m| \leq \sum_{j \neq m} |a_{mj}||v_m|.$$

Because $|v_m| > 0$, dividing both sides by $|v_m|$ gives

$$|\lambda||a_{mm}| \leq \sum_{j \neq m} |a_{mj}|.$$

or equivalently,

$$|\lambda| \leq \frac{\sum_{j \neq m} |a_{mj}|}{|a_{mm}|}.$$

Finally, since A is strictly diagonally dominant, $|a_{mm}| > \sum_{j \neq m} |a_{mj}|$, which implies

$$|\lambda| \leq \frac{\sum_{j \neq m} |a_{mj}|}{|a_{mm}|} < 1.$$

Since λ was chosen as an arbitrary eigenvalue of T, every eigenvalue satisfies $|\lambda| < 1$, which means $\rho(T) < 1$. Therefore, the Jacobi Method converges for any initial guess when A is strictly diagonally dominant.

5 Gauss-Seidel method

The **Gauss–Seidel Method** is an iterative method for solving a system of linear equations $A\mathbf{x} = \mathbf{b}$. The main difference between the Gauss–Seidel Method and the Jacobi Method is when the newly computed values are used.

In the Jacobi Method, every component of $\mathbf{x}^{(k+1)}$ is computed using only the values from the previous iteration $\mathbf{x}^{(k)}$. In contrast, the Gauss–Seidel Method immediately uses each newly computed value in the same iteration.

5.1 Basic of Gauss–Seidel Method

Consider a system of three linear equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3. \end{aligned}$$

We first solve each equation for its corresponding unknown:

$$\begin{aligned} x_1 &= \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, \\ x_2 &= \frac{b_2 - a_{21}x_1 - a_{23}x_3}{a_{22}}, \\ x_3 &= \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}}. \end{aligned}$$

Starting from an initial guess

$$\mathbf{x}^{(0)} = \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix}.$$

First, compute $x_1^{(k+1)}$ using the value from the previous iteration:

$$x_1^{(k+1)} = \frac{b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)}}{a_{11}}.$$

Once $x_1^{(k+1)}$ has been obtained, we use it when compute $x_2^{(k+1)}$:

$$x_2^{(k+1)} = \frac{b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)}}{a_{22}}.$$

Finally, both newly computed values are used to calculate $x_3^{(k+1)}$:

$$x_3^{(k+1)} = \frac{b_3 - a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)}}{a_{33}}.$$

Therefore, within a single iteration, the values are updated sequentially:

$$\boxed{x_1^{(k+1)} \rightarrow x_2^{(k+1)} \rightarrow x_3^{(k+1)}}$$

The newly computed values are used immediately rather than waiting until the next iteration. This sequential update is the key feature of the Gauss–Seidel Method.

5.2 Worked Example

Consider the system

$$\begin{aligned} 4x_1 + x_2 + x_3 &= 7, \\ 2x_1 + 7x_2 + x_3 &= -4, \\ x_1 - 3x_2 + 12x_3 &= 15. \end{aligned}$$

Solving each equation for its corresponding variable gives

$$\begin{aligned} x_1 &= \frac{7 - x_2 - x_3}{4}, \\ x_2 &= \frac{-4 - 2x_1 - x_3}{7}, \\ x_3 &= \frac{15 - x_1 + 3x_2}{12}. \end{aligned}$$

Starting from an initial guess

$$\mathbf{x}^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

First Iteration :
First, calculate $x_1^{(1)}$:

$$x_1^{(1)} = \frac{7 - 0 - 0}{4} = 1.75$$

Next, use this newly computed value $x_1^{(1)}$ to calculate $x_2^{(1)}$:

$$x_2^{(1)} = \frac{-4 - 2(1.75) - 0}{7} = -1.07143.$$

Finally, use both newly computed values:

$$x_3^{(1)} = \frac{15 - (1.75) + 3(-1.07143)}{12} \approx 0.66786.$$

Therefore,

$$\mathbf{x}^{(1)} = \begin{bmatrix} 1.75 \\ -1.07143 \\ 0.66786 \end{bmatrix}.$$

Second Iteration :
Now we use the values from the first iteration:

$$x_1^{(2)} = \frac{7 - (-1.07143) - (0.66786)}{4} \approx 1.85089$$

then use $x_1^{(2)}$

$$x_2^{(2)} = \frac{-4 - 2(1.85089) - (0.66786))}{7} \approx -1.19566.$$

Finally, use both newly computed values:

$$x_3^{(2)} = \frac{15 - (1.85089) + 3(-1.19566)}{12} \approx 0.63078.$$

Thus,

$$\mathbf{x}^{(2)} = \begin{bmatrix} 1.85089 \\ -1.19566 \\ 0.63078 \end{bmatrix}.$$

After further iterations, the approximations converge to

$$\mathbf{x} \approx \begin{bmatrix} 1.8556 \\ -1.2147 \\ 0.7917 \end{bmatrix}.$$

5.3 Matrix formulation

Recall that the coefficient matrix can be decomposed as

$$A = D + L + U,$$

where D contains the diagonal entries of A, L contains the entries below the diagonal, and U contains the entries above the diagonal. Starting from $A\mathbf{x} = \mathbf{b}$, we obtain $(D+L+U)\mathbf{x} = \mathbf{b}$. Now, consider the system

$$\begin{aligned} 4x_1 + x_2 + x_3 &= 7, \\ 2x_1 + 7x_2 + x_3 &= -4, \\ x_1 - 3x_2 + 12x_3 &= 15. \end{aligned}$$

Starting from an initial guess $\mathbf{x}^{(0)}$ the first Gauss–Seidel iteration can be written as

$$\begin{aligned} 4x_1^{(1)} &= 7 - x_2^{(0)} - x_3^{(0)}, \\ 2x_1^{(1)} + 7x_2^{(1)} &= -4 - x_3^{(0)}, \\ x_1^{(1)} - 3x_2^{(1)} + 12x_3^{(1)} &= 15. \end{aligned}$$

Thus, we have

$$\begin{bmatrix} 4 & 0 & 0 \\ 2 & 7 & 0 \\ 1 & -3 & 12 \end{bmatrix} \begin{bmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{bmatrix} = \begin{bmatrix} 7 \\ -4 \\ 15 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1^{(0)} \\ x_2^{(0)} \\ x_3^{(0)} \end{bmatrix}.$$

Therefore, for a general $n \times n$ system, the Gauss–Seidel iteration is

$$(D + L)\mathbf{x}^{(k+1)} = \mathbf{b} - U\mathbf{x}^{(k)}.$$

Here, $(D+L)$ is a lower-triangular matrix with nonzero diagonal entries, so $\mathbf{x}^{(k+1)}$ can be obtained efficiently by forward substitution. Solving $\mathbf{x}^{(k+1)}$ gives

$$\mathbf{x}^{(k+1)} = (D + L)^{-1}(\mathbf{b} - U\mathbf{x}^{(k)}).$$

However, In a C++ implementation, we do not need to explicitly form $(D+L)^{-1}$ or perform a separate forward-substitution routine. Instead, the forward-substitution process is carried out directly by updating each component sequentially, as shown in the following example:

$$x_1^{(1)} = \frac{b_1 - a_{12}x_2^{(0)} - a_{13}x_3^{(0)}}{a_{11}},$$
$$x_2^{(1)} = \frac{b_2 - a_{21}x_1^{(1)} - a_{23}x_3^{(0)}}{a_{22}},$$
$$x_3^{(1)} = \frac{b_3 - a_{31}x_1^{(1)} - a_{32}x_2^{(1)}}{a_{33}}.$$

Thus, for a general $n \times n$ system, the Gauss-Seidel method can be written as

$$\boxed{x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij}x_j^{(k+1)} - \sum_{j>i} a_{ij}x_j^{(k)}}{a_{ii}}}.$$

5.4 Convergence of the Gauss-Seidel Method

Recall that the Gauss–Seidel iteration can be written as

$$\mathbf{x}^{(k+1)} = (D+L)^{-1}(\mathbf{b} - U\mathbf{x}^{(k)}).$$

This can be written in the standard iterative form

$$\mathbf{x}^{(k+1)} = T_{GS}\mathbf{x}^{(k)} + \mathbf{c},$$

where

$$T_{GS} = -(D+L)^{-1}U$$

is the Gauss–Seidel iteration matrix, and

$$\mathbf{c} = (D+L)^{-1}\mathbf{b}.$$

The convergence of the Gauss–Seidel method is characterized by the spectral radius of the iteration matrix T_{GS}.

For a stationary iterative method of the form

$$\mathbf{x}^{(k+1)} = T_{GS}\mathbf{x}^{(k)} + \mathbf{c},$$

the method converges to the exact solution for every initial guess $\mathbf{x}^{(0)}$ if and only if

$$\boxed{\rho(T_{GS}) < 1,}$$

where $\rho(T_{GS}) = \max_i |\lambda_i|$ is the spectral radius of T_{GS}. We used this convergence criterion for the Jacobi method as well.

Now, we consider **symmetric positive-definite (SPD) matrices**. A matrix $A \in \mathbb{R}^{n\times n}$ is called symmetric positive definite (SPD) if it satisfies two conditions.

1. **Symmetry** : $A^T = A$. That means $a_{ij} = a_{ji}$ for every i, j.

2. **Positive definiteness** : For every nonzero vector $\mathbf{x}$, $\mathbf{x}^T A\mathbf{x} > 0$.

Define $\mathbf{x}^*$ be the exact solution of $A\mathbf{x}^* = \mathbf{b}$. Define the error $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$. The Gauss–Seidel iteration is

$$(D + L)\mathbf{x}^{(k+1)} = \mathbf{b} - U\mathbf{x}^{(k)}.$$

Since the exact solution satisfies

$$(D + L)\mathbf{x}^* = \mathbf{b} - U\mathbf{x}^*,$$

subtracting the two equations gives

$$(D + L)(\mathbf{x}^{(k+1)} - \mathbf{x}^*) = -U(\mathbf{x}^{(k)} - \mathbf{x}^*).$$

Therefore,

$$(D + L)\mathbf{e}^{(k+1)} = -U\mathbf{e}^{(k)}.$$

This is exactly the error iteration

$$\boxed{\mathbf{e}^{(k+1)} = T_{GS}\mathbf{e}^{(k)}.}$$

Now define

$$\mathbf{d}^{(k)} = \mathbf{e}^{(k+1)} - \mathbf{e}^{(k)}.$$

Thus,

$$(D + L)\mathbf{e}^{(k+1)} = -U(\mathbf{e}^{(k+1)} - \mathbf{d}^{(k)}).$$

Rewriting the equation, we have

$$(D + L)\mathbf{e}^{(k+1)} + U\mathbf{e}^{(k+1)} = U\mathbf{d}^{(k)}$$

Since $A = D + L + U$, we obtain

$$A\mathbf{e}^{(k+1)} = U\mathbf{d}^{(k)}.$$

Because A is SPD, define

$$||\mathbf{e}||_A = \sqrt{\mathbf{e}^T A\mathbf{e}}.$$

Consider the difference of their squared

$$||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 = (\mathbf{e}^{(k)})^T A\mathbf{e}^{(k)} - (\mathbf{e}^{(k+1)})^T A\mathbf{e}^{(k+1)}.$$

Since $\mathbf{e}^{(k)} = \mathbf{e}^{(k+1)} - \mathbf{d}^{(k)}$, we get

$$(\mathbf{e}^{(k+1)} - \mathbf{d}^{(k)})^T A(\mathbf{e}^{(k+1)} - \mathbf{d}^{(k)}) - (\mathbf{e}^{(k+1)})^T A\mathbf{e}^{(k+1)} = -2(\mathbf{d}^{(k)})^T A\mathbf{e}^{(k+1)} + (\mathbf{d}^{(k)})^T A\mathbf{d}^{(k)}.$$

From $A\mathbf{e}^{(k+1)} = U\mathbf{d}^{(k)}$. Multiplying both sides on the left by $(\mathbf{d}^{(k)})^T$, we have

$$(\mathbf{d}^{(k)})^T A\mathbf{e}^{(k+1)} = (\mathbf{d}^{(k)})^T U\mathbf{d}^{(k)}.$$

Therefore,

$$||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 = -2(\mathbf{d}^{(k)})^T U\mathbf{d}^{(k)} + (\mathbf{d}^{(k)})^T A\mathbf{d}^{(k)}.$$

Since A is symmetric, we have $U = L^T$. Therefore,

$$(\mathbf{d}^{(k)})^T A\mathbf{d}^{(k)} = (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} + 2(\mathbf{d}^{(k)})^T U\mathbf{d}^{(k)}$$

So,

$$||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 = (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)}.$$

Since A is SPD, $a_{ii} > 0$. Therefore for any nonzero $\mathbf{d}^{(k)}$, $(\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} > 0$, which means

$$||\mathbf{e}^{(k)}||_A > ||\mathbf{e}^{(k+1)}||_A.$$

Recall $\mathbf{e}^{(k+1)} = T_{GS}\mathbf{e}^{(k)}$. We have

$$||\mathbf{e}^{(k+1)}||_A = ||T_{GS}\mathbf{e}^{(k)}||_A < ||\mathbf{e}^{(k)}||_A.$$

Thus, T_{GS} is contraction in the A-norm, which gives $||T_{GS}||_A < 1$.
Since the spectral radius satisfies $\rho(T_{GS}) < ||T_{GS}||_A$, we conclude that

$$\boxed{\rho(T_{GS}) < 1.}$$

5.5 Why This Argument Does Not Apply to Jacobi

For the Jacobi method, we know

$$D\mathbf{e}^{(k+1)} = -(L + U)\mathbf{e}^{(k)}.$$

Because A is symmetric, we have $U = L^T$. Therefore,

$$D\mathbf{e}^{(k+1)} = -(L + L^T)\mathbf{e}^{(k)}.$$

Now we define $\mathbf{d}^{(k)} = \mathbf{e}^{(k+1)} - \mathbf{e}^{(k)}$ as well. Since, $\mathbf{e}^{(k)} = \mathbf{e}^{(k+1)} - \mathbf{d}^{(k)}$. We have

$$D\mathbf{e}^{(k+1)} = -(L + L^T)(\mathbf{e}^{(k+1)} - \mathbf{d}^{(k)}).$$

Rewriting the equation, we obtain

$$(D + L + L^T)\mathbf{e}^{(k+1)} = (L + L^T)\mathbf{d}^{(k)}.$$

Since $A = D + L + U = D + L + L^T$. Thus,

$$A\mathbf{e}^{(k+1)} = (L + L^T)\mathbf{d}^{(k)}.$$

We now follow the same argument used for Gauss–Seidel. Calculating $||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2$.

$$||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 = -2(\mathbf{d}^{(k)})^T A\mathbf{e}^{(k+1)} + (\mathbf{d}^{(k)})^T A\mathbf{d}^{(k)}.$$

Since $A\mathbf{e}^{(k+1)} = (L + L^T)\mathbf{d}^{(k)}$, we have

$$\begin{aligned}
||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 &= -2(\mathbf{d}^{(k)})^T(L + L^T)\mathbf{d}^{(k)} + (\mathbf{d}^{(k)})^T(D + L + L^T)\mathbf{d}^{(k)} \\
&= -2(\mathbf{d}^{(k)})^T(L + L^T)\mathbf{d}^{(k)} + (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} + (\mathbf{d}^{(k)})^T(L + L^T)\mathbf{d}^{(k)} \\
&= (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} - (\mathbf{d}^{(k)})^T(L + L^T)\mathbf{d}^{(k)}.
\end{aligned}$$

Since $L + L^T = A - D$, thus,

$$\begin{aligned}
||\mathbf{e}^{(k)}||_A^2 - ||\mathbf{e}^{(k+1)}||_A^2 &= (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} - (\mathbf{d}^{(k)})^T(L + L^T)\mathbf{d}^{(k)} \\
&= (\mathbf{d}^{(k)})^T D\mathbf{d}^{(k)} - (\mathbf{d}^{(k)})^T(A - D)\mathbf{d}^{(k)} \\
&= (\mathbf{d}^{(k)})^T(2D - A)\mathbf{d}^{(k)}.
\end{aligned}$$

Unlike the Gauss–Seidel case, this gives

$$\|\mathbf{e}^{(k)}\|_A^2 - \|\mathbf{e}^{(k+1)}\|_A^2 = (\mathbf{d}^{(k)})^T(2D - A)\mathbf{d}^{(k)},$$

which $(2D - A)$ is not guaranteed to be positive-definite even if A itself is symmetric positive-definite (SPD). Therefore, the right-hand side is not necessarily positive, and we cannot conclude that

$$||\mathbf{e}^{(k+1)}||_A^2 < ||\mathbf{e}^{(k)}||_A^2.$$

Thus, the Jacobi iteration matrix cannot in general be shown to be a contraction in the A-norm using this argument.

For example, consider the SPD matrix

$$A = \begin{pmatrix} 1 & 0.75 & 0.75 \\ 0.75 & 1 & 0.75 \\ 0.75 & 0.75 & 1 \end{pmatrix}.$$

Its eigenvalues are $(2.5, 0.25, 0.25)$, so A is positive-definite. However,

$$2D - A = \begin{pmatrix} 1 & -0.75 & -0.75 \\ -0.75 & 1 & -0.75 \\ -0.75 & -0.75 & 1 \end{pmatrix}$$

has eigenvalues $(-0.5, 1.75, 1.75)$, and is therefore not positive-definite. Hence, positive definiteness of A alone does not provide the same convergence proof for the Jacobi method. A different convergence condition, such as $\rho(T_J) < 1$, is required.

6 Conjugate Gradient method

We begin with the linear system $A\mathbf{x} = \mathbf{b}$, where $A \in \mathbb{R}^{n\times n}$ is symmetric positive definite.

The Conjugate Gradient (CG) method is an iterative method for solving such systems. Unlike Jacobi and Gauss–Seidel, which update the components of the solution vector directly, CG constructs a sequence of carefully chosen search directions.

Consider the quadratic function

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}\mathbf{x}.$$

Because A is symmetric,

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}.$$

Note:

$$d(\frac{1}{2}\mathbf{x}^T A\mathbf{x}) = \frac{1}{2}((d\mathbf{x})^T A\mathbf{x} + \mathbf{x}^T Ad\mathbf{x}).$$

Because a scalar equals its own transpose, we can transpose the first term

$$((d\mathbf{x})^T A\mathbf{x})^T = \mathbf{x}^T A^T d\mathbf{x}.$$

Substituting this back gives

$$\begin{aligned} d(\frac{1}{2}\mathbf{x}^T A\mathbf{x}) &= \frac{1}{2}(\mathbf{x}^T A^T d\mathbf{x} + \mathbf{x}^T Ad\mathbf{x}) \\ &= \frac{1}{2}\mathbf{x}^T (A + A^T)d\mathbf{x} \\ &= \frac{1}{2}\mathbf{x}^T (2A)d\mathbf{x} \\ &= A\mathbf{x}. \end{aligned}$$

Therefore, a stationary point of f satisfies

$$\nabla f(\mathbf{x}) = 0,$$

which is equivalent to $A\mathbf{x} = \mathbf{b}$. Since A is positive definite, $f(\mathbf{x})$ is strictly convex. Hence, this stationary point is the unique global minimum of f. Therefore, solving $A\mathbf{x} = \mathbf{b}$ is equivalent to finding the minimum of

$$\frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}\mathbf{x}.$$

The minimization problem can be viewed geometrically. Starting from an initial guess $\mathbf{x}^{(0)}$, we seek a sequence of approximations that gradually approaches the minimizer of $f(\mathbf{x})$. At each iteration, we move from the current approximation along a carefully chosen search direction.

6.1 Conjugate Gradient Iteration

We want to solve the linear system $A\mathbf{x} = \mathbf{b}$. Choose an initial approximation $\mathbf{x}^{(0)}$. The initial residual is

$$\mathbf{r}^{(0)} = \mathbf{b} - A\mathbf{x}^{(0)}.$$

The residual measures how closely the current approximation satisfies the original system. If $\mathbf{r}^{(0)} = \mathbf{0}$, we already have the exact solution. We initialize the first search direction as

$$\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$$

and the residual is the negative gradient of the quadratic function

$$\mathbf{r}^{(k)} = -\nabla f(\mathbf{x}^{(k)}).$$

Therefore, the residual points in the direction of steepest local decrease. CG uses it to construct its first search direction.

We update the approximation along the current search direction as follows:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}.$$

The step size α_k is chosen to minimize the objective function along the direction $\mathbf{p}^{(k)}$. Define

$$\phi(\alpha) = f\left(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}\right).$$

By the chain rule,

$$\phi'(\alpha) = \nabla f\left(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}\right)^T \mathbf{p}^{(k)}.$$

Recall that

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}.$$

Therefore,

$$\begin{aligned}
\phi'(\alpha) &= \left[A\left(\mathbf{x}^{(k)} + \alpha \mathbf{p}^{(k)}\right) - \mathbf{b}\right]^T \mathbf{p}^{(k)} \\
&= \left(A\mathbf{x}^{(k)} - \mathbf{b} + \alpha A\mathbf{p}^{(k)}\right)^T \mathbf{p}^{(k)} \\
&= \left(-\mathbf{r}^{(k)} + \alpha A\mathbf{p}^{(k)}\right)^T \mathbf{p}^{(k)} \\
&= -(\mathbf{r}^{(k)})^T \mathbf{p}^{(k)} + \alpha (\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}.
\end{aligned}$$

To minimize $\phi(\alpha)$, set $\phi'(\alpha_k) = 0$. Solving for α_k gives

$$\alpha_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}.$$

Since A is positive definite and $\mathbf{p}^{(k)} \neq \mathbf{0}$, the denominator is positive. Hence, $\phi(\alpha)$ is a strictly convex quadratic function of α, and this step size gives its unique minimum.

After determining the step size α_k, we update the approximation by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}.$$

The new residual is defined as

$$\mathbf{r}^{(k+1)} = \mathbf{b} - A\mathbf{x}^{(k+1)}.$$

Substituting the update formula, we obtain

$$\begin{aligned}\mathbf{r}^{(k+1)} &= \mathbf{b} - A\left(\mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}\right) \\ &= \mathbf{b} - A\mathbf{x}^{(k)} - \alpha_k A\mathbf{p}^{(k)} \\ &= \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)}.\end{aligned}$$

Therefore, the residual update is

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)}.$$

The next direction combines the new residual with the previous search direction

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}.$$

The coefficient β_k is chosen so that consecutive search directions are conjugate with respect to A

$$(\mathbf{p}^{(k+1)})^T A\mathbf{p}^{(k)} = 0.$$

Substituting the search direction update gives

$$\left(\mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}\right)^T A\mathbf{p}^{(k)} = 0.$$

Expanding and rearranging, we obtain

$$(\mathbf{r}^{(k+1)})^T A\mathbf{p}^{(k)} + \beta_k (\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)} = 0.$$

Since A is positive definite and $\mathbf{p}^{(k)} \neq \mathbf{0}$, the denominator is positive. Therefore,

$$\beta_k = -\frac{(\mathbf{r}^{(k+1)})^T A\mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}.$$

Using the residual update formula,

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)},$$

we obtain

$$A\mathbf{p}^{(k)} = \frac{\mathbf{r}^{(k)} - \mathbf{r}^{(k+1)}}{\alpha_k}.$$

Recall that

$$\phi'(\alpha_k) = -(\mathbf{r}^{(k)})^T \mathbf{p}^{(k)} + \alpha_k (\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)} = 0.$$

Therefore

$$(\mathbf{r}^{(k)})^T \mathbf{p}^{(k)} = \alpha_k (\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}.$$

On the other hand,

$$\begin{aligned}(\mathbf{r}^{(k+1)})^T\mathbf{p}^{(k)} &= (\mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)})^T\mathbf{p}^{(k)} \\ &= (\mathbf{r}^{(k)})^T\mathbf{p}^{(k)} - \alpha_k(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)} \\ &= 0.\end{aligned}$$

We have

$$\boxed{(\mathbf{r}^{(k+1)})^T\mathbf{p}^{(k)} = 0.}$$

This shows that the new residual is orthogonal to the previous search direction. Using the search direction update.

$$\mathbf{p}^{(k)} = \mathbf{r}^{(k)} + \beta_{k-1}\mathbf{p}^{(k-1)}.$$

We obtain

$$\begin{aligned}(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k)} &= (\mathbf{r}^{(k+1)})^T\left(\mathbf{p}^{(k)} - \beta_{k-1}\mathbf{p}^{(k-1)}\right) \\ &= \mathbf{r}^{(k+1)})^T\mathbf{p}^k - \beta_{k-1}(\mathbf{r}^{(k+1)})^T\mathbf{p}^{(k-1)} \\ &= -\beta_{k-1}(\mathbf{r}^{(k+1)})^T\mathbf{p}^{(k-1)}.\end{aligned}$$

By the A-conjugacy of consecutive search directions

$$(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k-1)} = 0,$$

we have

$$\begin{aligned}(\mathbf{r}^{(k+1)})^T\mathbf{p}^{(k-1)} &= (\mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)})^T\mathbf{p}^{(k-1)} \\ &= (\mathbf{r}^{(k)})^T\mathbf{p}^{(k-1)} - \alpha_k(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k-1)} \\ &= 0.\end{aligned}$$

Therefore,

$$\boxed{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k)} = 0.}$$

We can now simplify the numerator of the expression for β_k. Substituting the residual update into the numerator gives

$$\begin{aligned}(\mathbf{r}^{(k+1)})^T A\mathbf{p}^{(k)} &= \frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k)} - (\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{\alpha_k} \\ &= -\frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{\alpha_k}.\end{aligned}$$

Similarly, the denominator becomes

$$\begin{aligned}(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)} &= \frac{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)} - (\mathbf{p}^{(k)})^T\mathbf{r}^{(k+1)}}{\alpha_k} \\ &= \frac{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)}}{\alpha_k}.\end{aligned}$$

Since $\mathbf{p}^{(k)} = \mathbf{r}^{(k)} + \beta_{k-1}\mathbf{p}^{(k-1)}$, we have

$$
\begin{aligned}
(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)} &= (\mathbf{r}^{(k)} + \beta_{k-1}\mathbf{p}^{(k-1)})\mathbf{r}^{(k)} \\
&= (\mathbf{r}^{(k)})^T\mathbf{r}^{(k)} + \beta_{k-1}(\mathbf{r}^{k})^T\mathbf{p}^{(k-1)} \\
&= (\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}.
\end{aligned}
$$

Therefore,

$$
\boxed{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)} = (\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}.}
$$

Substituting these expressions into the formula for β_k, we obtain

$$
\beta_k = -\frac{(\mathbf{r}^{(k+1)})^T A\mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}} = -\frac{-\frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{\alpha_k}}{\frac{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)}}{\alpha_k}} = \frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}.
$$

Then

$$
\alpha_k = \frac{(\mathbf{r}^{(k)})^T\mathbf{p}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}} = \frac{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}.
$$

Thus, the standard formula for the CG coefficient is

$$
\boxed{\alpha_k = \frac{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}}
$$

$$
\boxed{\beta_k = \frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}}
$$

Combining the results derived above, we obtain the standard Conjugate Gradient (CG) iteration for solving the symmetric positive definite linear system $A\mathbf{x} = \mathbf{b}$. Starting from an initial approximation $\mathbf{x}^{(0)}$, the method iteratively updates the approximate solution, residual, and search direction as follows.

$$
\boxed{
\begin{aligned}
\mathbf{r}^{(0)} &= \mathbf{b} - A\mathbf{x}^{(0)}, \\
\mathbf{p}^{(0)} &= \mathbf{r}^{(0)}, \\
\alpha_k &= \frac{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{p}^{(k)})^T A\mathbf{p}^{(k)}}, \\
\mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \alpha_k\mathbf{p}^{(k)}, \\
\mathbf{r}^{(k+1)} &= \mathbf{r}^{(k)} - \alpha_k A\mathbf{p}^{(k)}, \\
\beta_k &= \frac{(\mathbf{r}^{(k+1)})^T\mathbf{r}^{(k+1)}}{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}, \\
\mathbf{p}^{(k+1)} &= \mathbf{r}^{(k+1)} + \beta_k\mathbf{p}^{(k)}.
\end{aligned}
}
$$

6.2 Compared with the Gradient Descent Method

The Gradient Descent method moves in the direction of the negative gradient, which is the direction of steepest local decrease. Define the residual at iteration k as

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}.$$

Since

$$-\nabla f(\mathbf{x}^{(k)}) = \mathbf{b} - A\mathbf{x}^{(k)} = \mathbf{r}^{(k)},$$

The approximate solution is updated by

$$\boxed{\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}}$$

where α_k is the step size, which determines how far the method moves along the current search direction. Define

$$\phi(\alpha) = f\left(\mathbf{x}^{(k)} + \alpha \mathbf{r}^{(k)}\right).$$

Differentiating with respect to α gives

$$\phi'(\alpha) = \nabla f\left(\mathbf{x}^{(k)} + \alpha \mathbf{r}^{(k)}\right)^T \mathbf{r}^{(k)}.$$

Using $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}$, we obtain

$$\begin{aligned}
\phi'(\alpha) &= (A\mathbf{x}^{(k)} + \alpha A\mathbf{r}^{(k)} - \mathbf{b})^T \mathbf{r}^{(k)} \\
&= (-\mathbf{r}^{(k)} + \alpha A\mathbf{r}^{(k)})^T \mathbf{r}^{(k)} \\
&= -(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)} + \alpha (\mathbf{r}^{(k)})^T A\mathbf{r}^{(k)}.
\end{aligned}$$

Setting $\phi'(\alpha_k) = 0$, we obtain

$$\boxed{\alpha_k = \frac{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^T A\mathbf{r}^{(k)}}}$$

Since A is positive definite and $\mathbf{r}^{(k)} \neq \mathbf{0}$, the denominator is positive. Therefore, $\phi(\alpha)$ is a strictly convex quadratic function, and this step size gives its unique minimum along the current search direction.

After updating the approximate solution, the new residual is

$$\begin{aligned}
\mathbf{r}^{(k+1)} &= \mathbf{b} - A\mathbf{x}^{(k+1)} \\
&= \mathbf{b} - A(\mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}) \\
&= \mathbf{r}^{(k)} - \alpha_k A\mathbf{r}^{(k)}.
\end{aligned}$$

The process repeats until the residual norm satisfies a prescribed convergence criterion.

Both **Gradient Descent** and **Conjugate Gradient** minimize the same quadratic objective function. However, they construct their search directions differently.

1. Gradient Descent uses only the current residual:

$$\mathbf{p}^{(k)} = \mathbf{r}^{(k)}.$$

2. Conjugate Gradient incorporates the previous search direction:

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}, \text{ where } \beta_k = \frac{(\mathbf{r}^{(k+1)})^T \mathbf{r}^{(k+1)}}{(\mathbf{r}^{(k)})^T \mathbf{r}^{(k)}}.$$

This additional term enables the CG method to construct mutually A-conjugate search directions. Gradient Descent is conceptually simple and provides the foundation for understanding CG. The key improvement introduced by CG is the use of previous search-direction information to accelerate convergence.

A Appendix

A.1 Symmetric Matrices Have Real Eigenvalues

A fundamental property of real symmetric matrices is that all of their eigenvalues are real. This result is important when analyzing the eigenvalues of iteration matrices and the spectral radius.

Theorem
Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix, so that

$$A^T = A.$$

Then every eigenvalue of A is real.

Proof.
Suppose that λ is an eigenvalue of A with corresponding nonzero eigenvector $\mathbf{v} \in \mathbb{C}^n$. Thus,

$$A\mathbf{v} = \lambda\mathbf{v}.$$

Although A is real, we allow $\mathbf{v}$ and λ to be complex. Take the conjugate transpose of $\mathbf{v}$ and multiply the eigenvalue equation on the left by $\mathbf{v}^*$:

$$\mathbf{v}^*A\mathbf{v} = \lambda\mathbf{v}^*\mathbf{v}.$$

Since A is real symmetric,

$$A^* = A^T = A.$$

Therefore,

$$(\mathbf{v}^*A\mathbf{v})^* = \mathbf{v}^*A^*\mathbf{v} = \mathbf{v}^*A\mathbf{v}.$$

Hence $\mathbf{v}^*A\mathbf{v}$ is real. Also,

$$\mathbf{v}^*\mathbf{v} = \sum_{i=1}^{n} |v_i|^2 > 0,$$

because $\mathbf{v} \neq \mathbf{0}$. From

$$\mathbf{v}^*A\mathbf{v} = \lambda\mathbf{v}^*\mathbf{v},$$

we can therefore write

$$\lambda = \frac{\mathbf{v}^*A\mathbf{v}}{\mathbf{v}^*\mathbf{v}}.$$

The numerator is real and the denominator is positive and real. Consequently,

$$\boxed{\lambda \in \mathbb{R}}.$$

Thus, every eigenvalue of a real symmetric matrix is real.

A.2 Positive Eigenvalues of Symmetric Positive Definite Matrices

If A is not only symmetric but also positive definite, we can obtain a stronger result.

Theorem. Let $A \in \mathbb{R}^{n \times n}$ be symmetric positive definite (SPD), meaning

$$A^T = A$$

and

$$\mathbf{x}^T A \mathbf{x} > 0 \qquad \text{for every } \mathbf{x} \neq \mathbf{0}.$$

Then every eigenvalue of A is strictly positive.

Proof.

Let λ be an eigenvalue of A with corresponding eigenvector $\mathbf{v} \neq \mathbf{0}$. Since A is symmetric, the previous result tells us that λ is real. From

$$A\mathbf{v} = \lambda \mathbf{v},$$

multiply both sides on the left by $\mathbf{v}^T$:

$$\mathbf{v}^T A \mathbf{v} = \lambda \mathbf{v}^T \mathbf{v}.$$

Because A is positive definite and $\mathbf{v} \neq \mathbf{0}$,

$$\mathbf{v}^T A \mathbf{v} > 0.$$

Also,

$$\mathbf{v}^T \mathbf{v} = \|\mathbf{v}\|_2^2 > 0.$$

Therefore,

$$\lambda = \frac{\mathbf{v}^T A \mathbf{v}}{\mathbf{v}^T \mathbf{v}} > 0.$$

Hence,

$$\boxed{\lambda > 0}.$$

Thus, a symmetric positive definite matrix has only positive real eigenvalues.

A.3 Converse: Positive Eigenvalues Imply Positive Definiteness

The previous result showed that if A is symmetric positive definite, then all of its eigenvalues are positive. For symmetric matrices, the converse is also true.

Theorem. Let $A \in \mathbb{R}^{n\times n}$ be symmetric. If all eigenvalues of A are positive, then A is positive definite.

Proof. Since A is symmetric, it can be orthogonally diagonalized by Spectral Theorem. Therefore, there exists an orthogonal matrix Q such that

$$A = Q\Lambda Q^T,$$

where

$$\Lambda = \begin{pmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{pmatrix}$$

and all eigenvalues satisfy

$$\lambda_i > 0.$$

For any nonzero vector $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{x}^T A\mathbf{x} = \mathbf{x}^T(Q\Lambda Q^T)\mathbf{x}.$$

Let

$$\mathbf{y} = Q^T\mathbf{x}.$$

Since Q is orthogonal,

$$\mathbf{x} \neq \mathbf{0} \quad \Longrightarrow \quad \mathbf{y} \neq \mathbf{0}.$$

Thus,

$$\mathbf{x}^T A\mathbf{x} = \mathbf{y}^T\Lambda\mathbf{y} = \sum_{i=1}^{n} \lambda_i y_i^2.$$

Because $\lambda_i > 0$ for every i and $\mathbf{y} \neq \mathbf{0}$, at least one y_i is nonzero. Hence,

$$\sum_{i=1}^{n} \lambda_i y_i^2 > 0.$$

Therefore,

$$\mathbf{x}^T A\mathbf{x} > 0 \qquad \text{for every } \mathbf{x} \neq \mathbf{0}.$$

Hence, A is positive definite.